

Data Base Analysis — Referência Técnica

Importação, limpeza e pipeline SQL→R para dados de nascidos vivos (SINASC/DataSUS)

```
knitr::opts_chunk$set(echo = TRUE, eval = TRUE,  
message = FALSE, warning = FALSE,  
fig.width = 10, fig.height = 6)
```

1. Visão Geral

Pipeline completo de importação, limpeza e análise de dados de nascidos vivos brasileiros (SINASC/DataSUS) utilizando R, SQL e SQLite.

Objetivo: Diagnosticar e resolver problemas de qualidade em dados reais, aplicando técnicas de limpeza e validação que garantem a integridade analítica dos resultados.

1.1 Conjunto de Dados

Duas tabelas que formam um modelo relacional de nascidos vivos entre 2019 e 2023:

- `nascimentos.csv` — Tabela-fato com problemas de qualidade intencionais. Chave primária: `ID_NASCIMENTO`

- `uf_referencia.csv` — Dimensão geográfica (UF, REGIAO). UF é a chave estrangeira

Dicionário de Variáveis

Variável	Tipo	Descrição
<code>`ID_NASCIMENTO`</code>	<code>int</code>	Identificador único (PK)
<code>`ANO`</code>	<code>int</code>	Ano do nascimento (2019–2023)
<code>`UF`</code>	<code>chr</code>	Sigla do estado (FK → <code>`uf_referencia`</code>)
<code>`SEXO`</code>	<code>chr</code>	Sexo do bebê (texto com grafias variadas)
<code>`IDADE_MAE`</code>	<code>int</code>	Idade da mãe em anos
<code>`PESO_GRAMAS`</code>	<code>int</code>	Peso ao nascer, em gramas
<code>`SEMANAS_GESTACAO`</code>	<code>int</code>	Idade gestacional em semanas
<code>`TIPO_PARTO`</code>	<code>int</code>	1 = Vaginal, 2 = Cesáreo
<code>`CONSULTAS_PRENATAL`</code>	<code>int</code>	Número de consultas de pré-natal
<code>`APGAR5`</code>	<code>int</code>	Apgar no 5º minuto (0 a 10)

2. Importação e Diagnóstico

2.1 Pacotes e Importação

```
library(dplyr)
library(skimr)
library(visdat)
library(kableExtra)
library(DBI)
library(RSQLite)
library(naniar)
library(ggplot2)
library(knitr)
nascimentos <- read.csv("nascimentos.csv",
stringsAsFactors = FALSE)
uf_ref <- read.csv("uf_referencia.csv", stringsAsFactors =
FALSE)
cat("Nascimentos:", nrow(nascimentos), "linhas e",
ncol(nascimentos), "colunas\n")
cat("UF Referência:", nrow(uf_ref), "linhas e",
ncol(uf_ref), "colunas\n")
```

2.2 Diagnóstico de Qualidade

```
str(nascimentos)
skim(nascimentos)
```

Mapa de Valores Ausentes

```
vis_miss(nascimentos) +
theme(
axis.text.x = element_text(size = 7, angle = 45, hjust =
1)
)
```

Observações

- PESO_GRAMAS: Importada como texto (character) devido a ruídos ou espaços em branco
 - SEXO: Variável qualitativa com grafias inconsistentes (ex: "fem", "masc", "f")
 - IDADE_MAE, PESO_GRAMAS, SEMANAS_GESTACAO, TIPO_PARTO: Apresentam registros vazios (NAs)
-

3. Limpeza com dplyr

3.1 SEXO: Grafia Inconsistente

```
# Diagnóstico das grafias
nascimentos |> count(SEX0)
# Padronização
nascimentos <- nascimentos |>
mutate(
  SEX0 = case_when(
    SEX0 %in% c("M", "m", "Masculino", "masc", "MASCULINO") ~
      "Masculino",
    SEX0 %in% c("F", "f", "Feminino", "FEMININO", "fem") ~
      "Feminino",
    TRUE ~ NA_character_
  )
)
nascimentos |> count(SEX0)
```

3.2 PESO_GRAMAS: Conversão e Validação

```
nascimentos <- nascimentos |>
mutate(PESO_GRAMAS =
  suppressWarnings(as.numeric(PESO_GRAMAS)))
cat("NA:", sum(is.na(nascimentos$PESO_GRAMAS)), "\n")
cat("Negativos:", sum(nascimentos$PESO_GRAMAS < 0, na.rm =
  TRUE), "\n")
# Filtrar apenas registros com peso válido e positivo
nascimentos <- nascimentos |>
filter(!is.na(PESO_GRAMAS) & PESO_GRAMAS > 0)
cat("Linhas após tratar PESO_GRAMAS:", nrow(nascimentos),
  "\n")
```

3.3 IDADE_MAE: Faixa Biológica Plausível

```
# Critério: idade entre 10 e 55 anos (faixa biologicamente plausível)
nascimentos |>
filter(is.na(IDADE_MAE) | IDADE_MAE < 10 | IDADE_MAE > 55)
|>
count(IDADE_MAE)
nascimentos <- nascimentos |>
filter(!is.na(IDADE_MAE) & IDADE_MAE >= 10 & IDADE_MAE <= 55)
cat("Linhas após tratar IDADE_MAE:", nrow(nascimentos),
    "\n")
```

3.4 UF: Siglas Inválidas

```
ufs_validas <- uf_ref$UF
# Siglas inválidas encontradas
nascimentos |> filter(!UF %in% ufs_validas) |> select(UF)
|> unique()
# Remoção
nascimentos <- nascimentos |>
filter(UF %in% ufs_validas)
cat("Linhas após tratar UF:", nrow(nascimentos), "\n")
```

3.5 Registros Duplicados

```
cat("Duplicadas:", sum(duplicated(nascimentos)), "\n")
nascimentos <- nascimentos |> distinct()
cat("Linhas após remover duplicatas:", nrow(nascimentos),
"\n")
```

Resumo da Limpeza

```
| Etapa | Registros Removidos | Motivo |
| PESO_GRAMAS | 183 | Valores não numéricos, negativos ou ausentes |
| IDADE_MAE | 198 | Idades fora da faixa biológica (10-55 anos) |
| UF | 114 | Siglas de estados inválidas |
| Duplicatas | 75 | Registros completamente duplicados |
| **Total** | **570** | **9,4% do dataset original** |
```

4. Variáveis Derivadas

4.1 Rótulo de Tipo de Parto

```
nascimentos <- nascimentos |>
mutate(TIPO_PARTO_LABEL = if_else(TIPO_PARTO == 1,
"Vaginal", "Cesáreo", missing = NA_character_))
nascimentos |> count(TIPO_PARTO, TIPO_PARTO_LABEL)
```

4.2 Baixo Peso e Prematuridade

```
nascimentos <- nascimentos |>
mutate(
  BAIXO_PESO = if_else(PESO_GRAMAS < 2500, "Sim", "Nao"),
  PREMATURO = if_else(SEMANAS_GESTACAO < 37, "Sim", "Nao",
missing = "Nao")
)
nascimentos |> count(BAIXO_PESO)
nascimentos |> count(PREMATURO)
```

4.3 Faixa Etária da Mãe

```
nascimentos <- nascimentos |>
mutate(
  FAIXA_IDADE_MAE = case_when(
    IDADE_MAE < 20 ~ "Adolescente",
    IDADE_MAE >= 20 & IDADE_MAE <= 34 ~ "Adulta",
    IDADE_MAE >= 35 ~ "35 ou mais",
    TRUE ~ NA_character_
  )
)
nascimentos |> count(FAIXA_IDADE_MAE)
```


5. Análise Exploratória com dplyr

5.1 Indicadores por Região

```
nascimentos |>
inner_join(uf_ref, by = "UF") |>
group_by(REGIAO) |>
summarise(
  n = n(),
  pct_cesarea = round(mean(TIPO_PARTO == 2) * 100, 1),
  pct_baixopeso = round(mean(BAIXO_PESO == "Sim") * 100, 1),
  pct_prematuro = round(mean(PREMATURO == "Sim") * 100, 1),
  .groups = "drop"
) |>
arrange(desc(pct_cesarea))
```

5.2 Tabela Consolidada

```

resumo <- data.frame(
  Indicador = c("Nascimentos", "Peso médio (g)", "%
  Cesárias", "% Baixo peso"),
  Valor = c(
    nrow(nascimentos),
    round(mean(nascimentos$PESO_GRAMAS), 0),
    round(mean(nascimentos$TIPO_PARTO == 2, na.rm = TRUE) *
    100, 1),
    round(mean(nascimentos$BAIXO_PESO == "Sim") * 100, 1)
  )
)
resumo |>
knitr::kable(
  format = "html",
  booktabs = TRUE,
  col.names = c("Indicador", "Valor"),
  align = c("l", "r"),
  caption = "Indicadores Consolidados de Nascimentos"
) |>
kable_styling(
  bootstrap_options = c("striped", "hover"),
  full_width = FALSE
)

```

5.3 Distribuição por Sexo

```

nascimentos |>
count(SEX0) |>
mutate(pct = round(n / sum(n) * 100, 1))

```

5.4 Evolução Temporal

```
nascimentos |>
group_by(ANO) |>
summarise(
  n = n(),
  pct_cesarea = round(mean(TIPO_PARTO == 2) * 100, 1),
  peso_medio = round(mean(PESO_GRAMAS), 0),
  .groups = "drop"
)
```

6. Pipeline SQL→R

6.1 Criando o Banco SQLite

```

con <- dbConnect(RSQLite::SQLite(), "nascimentos.sqlite")
# Criar tabela de referência
query_uf <- "CREATE TABLE IF NOT EXISTS uf_ref(
UF TEXT PRIMARY KEY,
REGIAO TEXT
);"
# Criar tabela principal com chave estrangeira
query_nascimentos <- "CREATE TABLE IF NOT EXISTS
nascimentos(
ID_NASCIMENTO INTEGER PRIMARY KEY,
ANO INTEGER,
UF TEXT,
SEXO TEXT,
IDADE_MAE INTEGER,
PESO_GRAMAS REAL,
SEMANAS_GESTACAO INTEGER,
TIPO_PARTO INTEGER,
CONSULTAS_PRENATAL INTEGER,
APGAR5 INTEGER,
TIPO_PARTO_LABEL TEXT,
BAIXO_PESO TEXT,
PREMATURO TEXT,
FAIXA_IDADE_MAE TEXT,
FOREIGN KEY(UF) REFERENCES uf_ref(UF)
);"
dbExecute(con, query_uf)
dbExecute(con, query_nascimentos)
# Inserir dados
dbWriteTable(conn = con, name = "uf_ref", value = uf_ref,
overwrite = TRUE)
dbWriteTable(conn = con, name = "nascimentos", value =
nascimentos, overwrite = TRUE)
# Listar tabelas
dbListTables(con)

```

6.2 SELECT com WHERE e ORDER BY

10 bebês prematuros com menor peso:

```
dbGetQuery(con, "  
SELECT ANO, UF, IDADE_MAE, PESO_GRAMAS  
FROM nascimentos  
WHERE PREMATURO = 'Sim'  
ORDER BY PESO_GRAMAS ASC  
LIMIT 10  
")
```

6.3 GROUP BY com HAVING

% de cesáreas por UF (apenas UFs com mais de 100 nascimentos):

```
dbGetQuery(con, "  
SELECT UF,  
COUNT(*) AS n,  
ROUND(AVG(CASE WHEN TIPO_PARTO = 2 THEN 1.0 ELSE 0 END) *  
100, 1) AS pct_cesarea  
FROM nascimentos  
GROUP BY UF  
HAVING n > 100  
ORDER BY pct_cesarea DESC  
")
```

6.4 JOIN entre Tabelas

Número de nascimentos por região:

```
dbGetQuery(con, "  
SELECT r.REGIAO, COUNT(*) AS n  
FROM nascimentos AS n_  
INNER JOIN uf_ref AS r  
ON n_.UF = r.UF  
GROUP BY r.REGIAO  
ORDER BY n DESC  
")
```

6.5 Encerramento

```
dbDisconnect(con)
```

7. Resultados e Interpretação

7.1 Resumo da Limpeza

- Registros iniciais: 6.080
- Registros finais: 5.510
- Total removidos: 570 (9,4%)

As exclusões ocorreram devido à presença de valores ausentes, valores inválidos e registros que não atendiam aos critérios de consistência definidos para a análise.

7.2 Consistência entre dplyr e SQL

As análises realizadas com dplyr e SQL produziram resultados equivalentes, demonstrando que ambas as linguagens permitem realizar as mesmas operações de manipulação e agregação de dados quando utilizadas corretamente.

- SQL: Consultas declarativas
- dplyr: Pipelines legíveis e modulares

7.3 Destaques

- A região Norte apresentou a maior proporção de cesáreas
- A idade materna predominante é na faixa Adulta (20–34 anos)
- A taxa de prematuridade varia entre regiões, sendo maior no Centro-Oeste

8. Sessão

```
sessionInfo()
```